Practical 3 – Advanced SQL on shopdb — views, indexing, CTEs and window functions

```
use shopdb;
# 1. Create a view named vw_order_value (order_id, customer, order_value) that gives each order's total value; query the view.

create or replace view vw_order_value as
select
        o.order_id,
        name as customer,
        sum(oi.quantity * p.price) as order_value
from orders o
join customers c
        on o.customer_id=c.customer_id
join order_items oi
        on o.order_id=oi.order_id
join products p
        on oi.product_id=p.product_id
group by
        o.order_id,
        c.name;

select * from vw_order_value;
```

OUTPUT:

order_id	customer	order_value
1	Asha Rao	64000.00
3	Asha Rao	2000.00
2	Ravi Nair	3800.00
4	Sara Ali	63000.00

```
# 2. Create an index on orders(customer_id)
create index idx_create_customer_id
on orders(customer_id);
show index from orders;
```

OUTPUT:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
orders	0	PRIMARY	1	order_id	A	5	NULL	NULL		BTREE			YES	NULL
orders	1	idx_create_customer_id	1	customer_id	A	3	NULL	NULL	YES	BTREE			YES	NULL

3. Using a Common Table Expression, compute the revenue for each month, then add a column showing the month-over-month change (use an appropriate window function).

```
with monthly_revenue as (
        select
                date_format(o.order_date, '%Y-%m') as month,
                sum(oi.quantity * p.price) as revenue
  from orders o
  join order_items oi
        on o.order_id = oi.order_id
  join products p
        on oi.product_id = p.product_id
  group by
        o.order_date
)
select
        month,
        revenue,
        lag(revenue) over (order by month) as previous_month_revenue,
        revenue - lag(revenue) over (order by month) as month_over_month_change
from monthly_revenue
order by month;
```

OUTPUT:

month	revenue	previous_month_revenue	month_over_month_change
2024-04-01	64000.00	NULL	NULL
2024-05-03	3800.00	64000.00	-60200.00
2024-06-10	2000.00	3800.00	-1800.00
2024-06-12	63000.00	2000.00	61000.00

4. Using a window function, rank the products by revenue within each category (ranking should restart for each category).

```
with product_revenue as(
        select
                c.category_id,
                c.category_name,
                p.product_id,
                p.product_name,
                sum(oi.quantity*p.price) as revenue
```

```
        from products p
        join order_items oi
                on p.product_id=oi.product_id
        join categories c
                on c.category_id=p.category_id
        group by
                p.category_id,
                p.product_id,
                p.product_name
)
select
        category_id,
        category_name,
        product_id,
        product_name,
        revenue,
        rank() over (
                partition by category_id
                order by category_name desc
        ) as revenue_rank
from product_revenue
order by
        category_id,
        revenue_rank;
```

OUTPUT:

category_id	category_name	product_id	product_name	revenue	revenue_rank
1	Electronics	1	Laptop	120000.00	1
1	Electronics	2	Headphones	6000.00	1
2	Books	3	SQL Book	2400.00	1
3	Clothing	5	T-Shirt	1400.00	1
3	Clothing	6	Jacket	3000.00	1